gcTMB

2026-11-12

The package gcTMB fits growth curves over time using the Template Model Builder (TMB) framework. The curves can depend on covariates, have random effects (e.g. for repeated measurements of the same individuals), and the response can follow a range of distributions.

Growth models

All models are of the form $f(t) = b_1 F\left(\frac{t-b_2}{b_3}\right)$, with a function F increasing from 0 to 1 that has its largest slope at 0:

- b_1 is the asymptote, the response approached at late times,
- b_2 is the time of the maximum growth rate, the inflection point of the curve,
- b_3 is the scale of the curve on the time axis, positive for an increasing curve.

The parameters b_4 and b_5 are reserved for future models; they are not used and always fixed.

logistic

$$f(t) = \frac{b_1}{1 + \exp\left(-\frac{t-b_2}{b_3}\right)}$$

The curve is symmetric around b_2, where it reaches half of its asymptote. The maximum growth rate is $b_1/(4b_3)$.

gompertz

$$f(t) = b_1 \exp\left(-\exp\left(-\frac{t-b_2}{b_3}\right)\right)$$

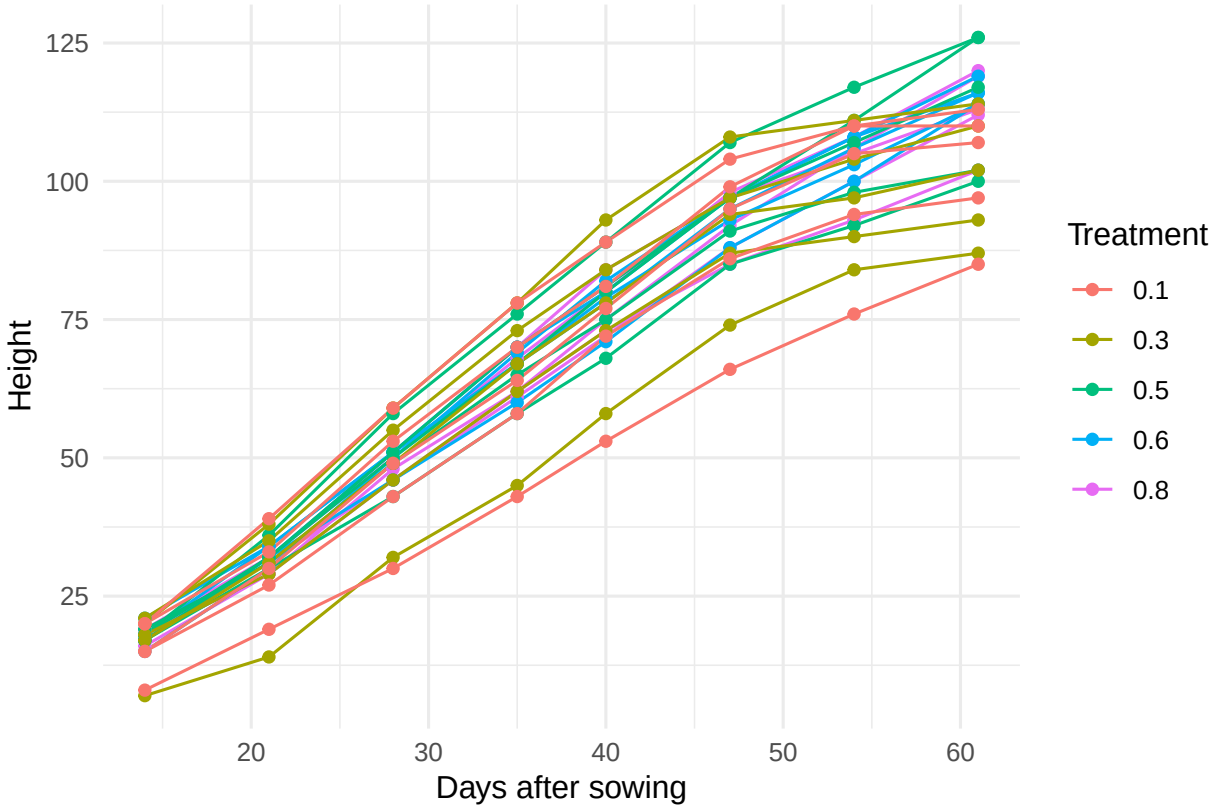
The curve reaches $e^{-1} \approx 37\%$ of its asymptote at b_2, and approaches the asymptote more slowly than it leaves zero. The maximum growth rate is $b_1/(e\, b_3)$.

The faba bean data

The data set `FabaHeights25` contains the heights of 25 faba bean plants, five at each of five treatment levels, measured on eight days between 14 and 61 days after sowing (`DAS`).

```
library(gcTMB)
library(ggplot2)
head(FabaHeights25)
#>   Tube_ID Treatment       Date DAS height DASfact plant_ID trt
#> 1       6       0.8 2025-09-24  14     19      14        6 0.8
#> 2       7       0.8 2025-09-24  14     18      14        7 0.8
```

```
#> 3       8       0.8 2025-09-24  14      18      14        8 0.8
#> 4       9       0.8 2025-09-24  14      16      14        9 0.8
#> 5      10       0.8 2025-09-24  14      17      14       10 0.8
#> 6      11       0.6 2025-09-24  14      17      14       11 0.6
ggplot(FabaHeights25, aes(x = DAS, y = height, group = plant_ID, colour = factor(Treatment))) +
  geom_line() +
  geom_point() +
  labs(x = "Days after sowing", y = "Height", colour = "Treatment") +
  theme_minimal()
```



Fitting a growth curve

The main function is `gcTMB()`, with the response and the time in the formula, and the growth model in `model`.

```
fit <- gcTMB(height ~ DAS, data = FabaHeights25, model = "logistic")
summary(fit)
#> Call:
#> gcTMB(form = height ~ DAS, data = FabaHeights25, model = "logistic")
#>
#> Model:  logistic, increasing curve
#> Family: gaussian, identity link for all parameters
#>
#> Number of observations: 200
#> Log-likelihood: -700.1 (df = 4), AIC: 1408, BIC: 1421
```

```
#>
#> Coefficients:
#>                Estimate Std. Error
#> b1:(Intercept)   116.63      2.577
#> b2:(Intercept)    32.32      0.698
#> b3:(Intercept)    10.97      0.540
#>
#> Residual standard deviation: 8.078 on 197 degrees of freedom
#>
#> Optimiser: relative convergence (4), maximum absolute gradient 2.5e-11
```

Starting values are found from the data by a linearisation of the curve: for a given asymptote b_1, $F^{-1}(y/b_1) = (t - b_2)/b_3$ is linear in time, so a linear regression of the transformed mean responses on time gives b_2 and b_3. The asymptote is profiled, as the observations often do not reach it yet. `gcTMBstart()` shows the starting values; the `start` argument provides starting values for some or all of the parameters, and the `fix` argument fixes parameters at these values.

```
gcTMBstart(height ~ DAS, data = FabaHeights25, model = "logistic")
#> b1:(Intercept) b2:(Intercept) b3:(Intercept) b4:(Intercept) b5:(Intercept)
#>      118.82055       33.02562       11.24957        0.00000        0.00000
```

The model is fitted by maximum likelihood. For gaussian (and lognormal) models without random effects, the inference follows `nls()` by default: the residual standard deviation is the square root of the unbiased residual variance with n - p residual degrees of freedom, and confidence and prediction intervals use t quantiles.

Model comparison

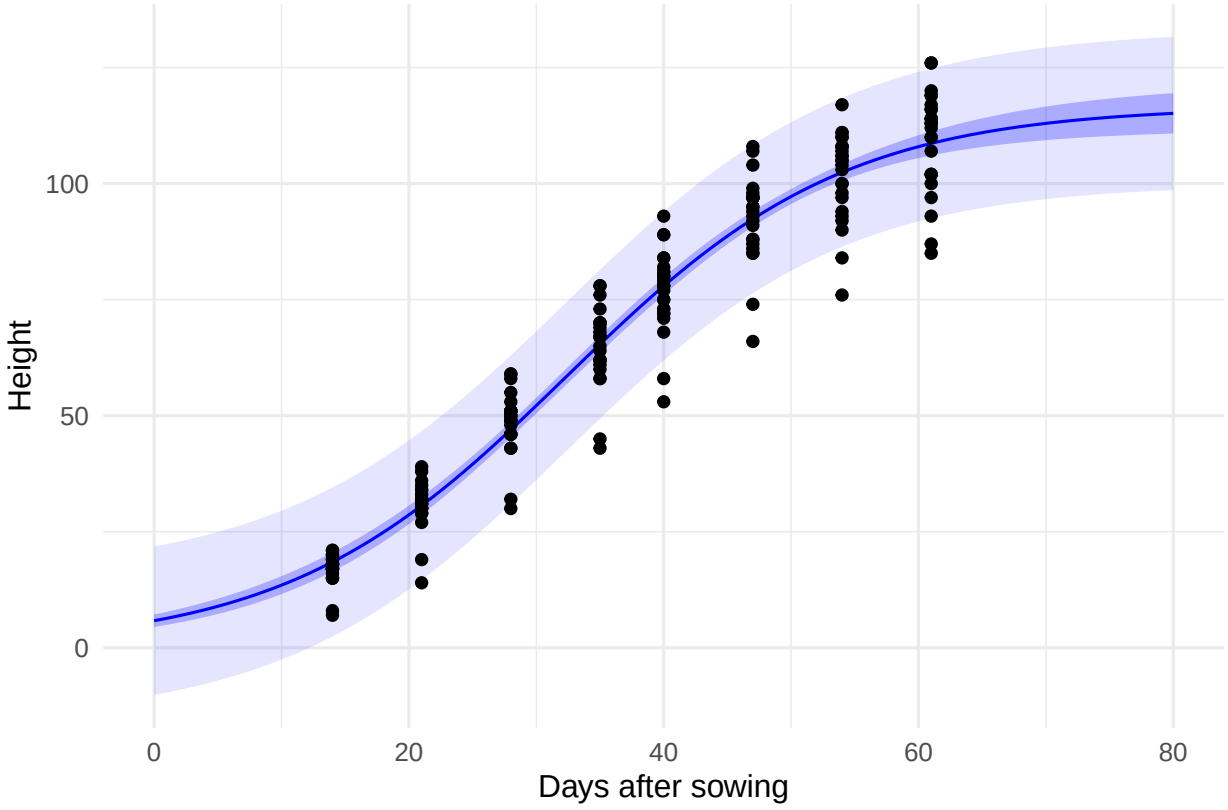
The logistic and Gompertz models are not nested, but can be compared by their AIC.

```
fitg <- gcTMB(height ~ DAS, data = FabaHeights25, model = "gompertz")
AIC(fit, fitg)
#>      df      AIC
#> fit   4 1408.214
#> fitg  4 1407.311
```

Predictions

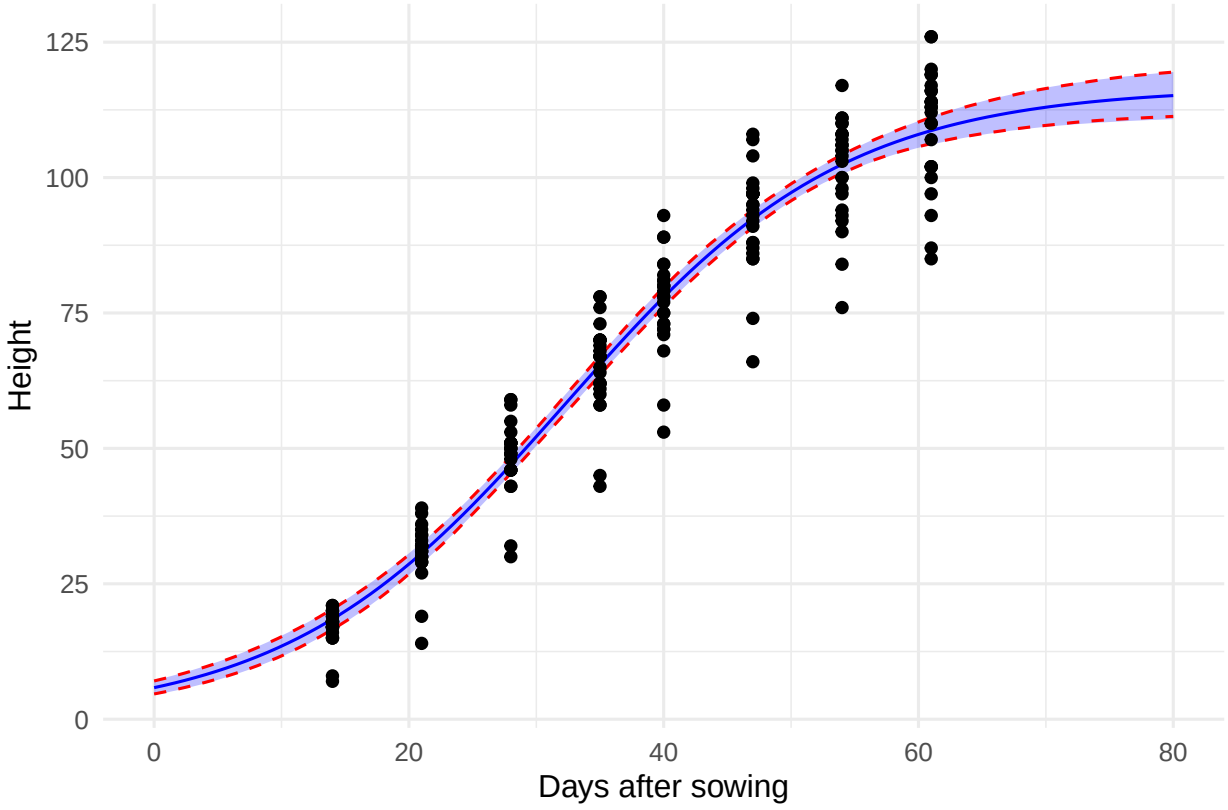
With `interval = "confidence"`, `predict()` adds standard errors and confidence intervals for the mean response by the delta method; with `interval = "prediction"`, the intervals are for a new observation.

```
nd <- data.frame(DAS = seq(0, 80, length.out = 100))
ci <- predict(fit, newdata = nd, interval = "confidence")
pi <- predict(fit, newdata = nd, interval = "prediction")
ggplot(FabaHeights25, aes(x = DAS, y = height)) +
  geom_ribbon(data = cbind(nd, pi), aes(x = DAS, ymin = lwr, ymax = upr), inherit.aes = FALSE, fill = "
  geom_ribbon(data = cbind(nd, ci), aes(x = DAS, ymin = lwr, ymax = upr), inherit.aes = FALSE, fill = "
  geom_line(data = cbind(nd, ci), aes(x = DAS, y = fit), colour = "blue") +
  geom_point() +
  labs(x = "Days after sowing", y = "Height") +
  theme_minimal()
```



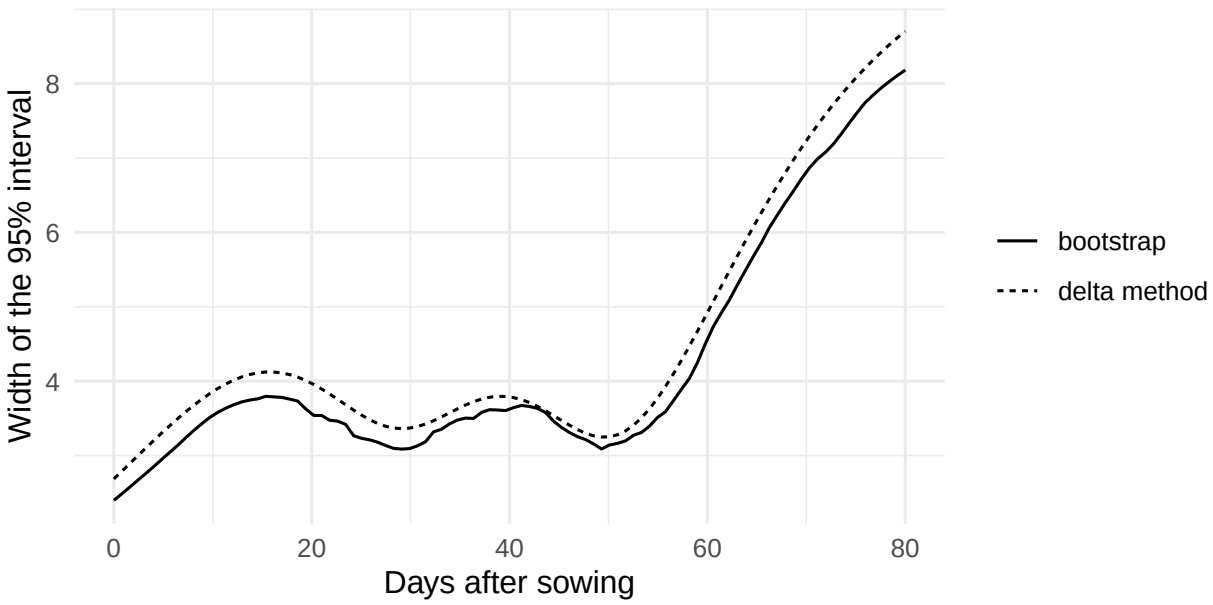
Bootstrap confidence intervals The delta method relies on the curve being close to linear in its parameters near the estimates. A parametric bootstrap avoids this: `bootstrap()` simulates new responses from the fitted model and fits the model again to each of them. Passed to `predict()` by the `boot` argument, the confidence intervals are the percentile intervals of the bootstrap curves, and `se.fit` is their standard deviation. The same bootstrap sample can be reused for the confidence intervals of the coefficients and of `timeto()`, without fitting the models again.

```
bt <- bootstrap(fit, nsim = 500, seed = 1, progress = FALSE)
cib <- predict(fit, newdata = nd, interval = "confidence", boot = bt)
# delta method: shaded; bootstrap: dashed limits
ggplot(FabaHeights25, aes(x = DAS, y = height)) +
  geom_ribbon(data = cbind(nd, ci), aes(x = DAS, ymin = lwr, ymax = upr), inherit.aes = FALSE, fill = "
  geom_line(data = cbind(nd, cib), aes(x = DAS, y = lwr), linetype = "dashed", colour = "red") +
  geom_line(data = cbind(nd, cib), aes(x = DAS, y = upr), linetype = "dashed", colour = "red") +
  geom_line(data = cbind(nd, ci), aes(x = DAS, y = fit), colour = "blue") +
  geom_point() +
  labs(x = "Days after sowing", y = "Height") +
  theme_minimal()
```



The two intervals nearly coincide here, as the curve is well determined by 200 observations. Their widths show the differences, which are largest beyond the observed days, where the curve depends on the extrapolated asymptote:

```
widths <- rbind(data.frame(nd, width = ci$upr - ci$lwr, method = "delta method"),
                data.frame(nd, width = cib$upr - cib$lwr, method = "bootstrap"))
ggplot(widths, aes(x = DAS, y = width, linetype = method)) +
  geom_line() +
  labs(x = "Days after sowing", y = "Width of the 95% interval", linetype = NULL) +
  theme_minimal()
```




```
confint(fit, boot = bt)
#>                      2.5 %    97.5 %
#> b1:(Intercept) 112.339181 121.78441
#> b2:(Intercept)  31.122156  33.72075
#> b3:(Intercept)  10.033456  11.99277
#> sigma            7.254138   8.79820
```

Scale or rate

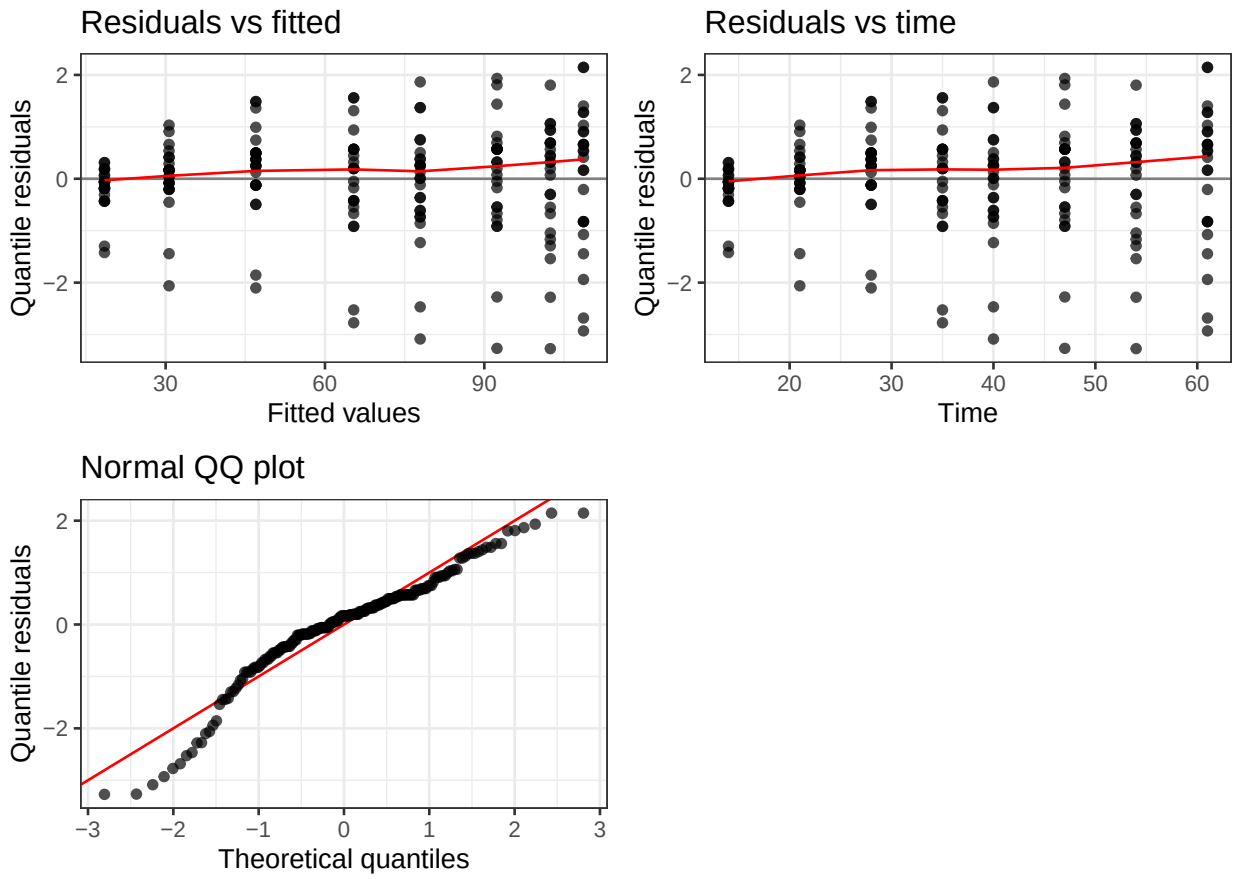
With the inverse link for b_3, `link = c(b3 = "inverse")`, the coefficient of b_3 is the rate $1/b_3$. Without covariates and random effects on b_3 the fit is the same; with them, covariates and random effects act additively on the rate instead of the scale.

```
fitr <- gcTMB(height ~ DAS, data = FabaHeights25, link = c(b3 = "inverse"))
coef(fitr)
#> b1:(Intercept) b2:(Intercept) b3:(Intercept)      log_sigma
#>   116.62848928    32.31782512     0.09113283     2.08159664
coef(fit)[["b3:(Intercept)"]]
#> [1] 10.97299
```

Diagnostics

`plot()` draws residuals against the fitted values and against time, and a normal QQ plot. The repeated measurements of the same plants are not independent; the random effects below account for that.

```
plot(fit, seed = 1)
```



Time to reach a level of the asymptote

`timeto()` gives the times at which a curve reaches a percentage of its asymptote, $b_2 + b_3 F^{-1}(p/100)$, with standard errors by automatic differentiation. With `type = "absolute"`, the times at which the curve reaches given responses, here a height of 50cm.

```
tt <- timeto(fit, respLev = c(10, 50, 90))
tt
#> Times to reach the percentages of the asymptote:
#>
#>         Estimate Std.Error
#> 10:lf1     8.21     0.835
#> 50:lf2    32.32     0.698
#> 90:lf3    56.43     1.757
timeto(fit, respLev = 50, type = "absolute")
#> Times to reach the responses:
#>
#>           Estimate Std.Error
#> y=50:lf1      29.2     0.327
```

`confint()` gives profile likelihood intervals by default, `method = "wald"` Wald intervals and `method = "boot"` parametric bootstrap intervals.

```
confint(tt)
#>             2.5 %    97.5 %
#> 10:lf1  6.503015  9.800857
#> 50:lf2 31.031673 33.819370
#> 90:lf3 53.229573 60.225189
confint(tt, method = "wald")
#>             2.5 %    97.5 %
#> 10:lf1  6.560855  9.854529
#> 50:lf2 30.941222 33.694428
#> 90:lf3 52.962176 59.893740
```

Covariates and random effects

Covariates on the parameters are given by `fform`, e.g. `b1 + b2 ~ trt` for effects of the treatment on the asymptote and the time of the maximum growth rate. Random effects are given by `rform`, e.g. `b1 + b2 ~ 1 | plant_ID` for correlated random effects of each plant on the asymptote and the time of the maximum growth rate, which account for the repeated measurements of the plants.

```
FabaHeights25$trt <- factor(FabaHeights25$Treatment)
fitre <- gcTMB(height ~ DAS, fform = b1 + b2 ~ trt, rform = b1 + b2 ~ 1 | plant_ID, data = FabaHeights25
summary(fitre)
#> Call:
#> gcTMB(form = height ~ DAS, fform = b1 + b2 ~ trt, rform = b1 +
#>     b2 ~ 1 | plant_ID, data = FabaHeights25)
#>
#> Model:  logistic, increasing curve
#> Family: gaussian, identity link for all parameters
#>
#> Number of observations: 200, groups: plant_ID 25
#> Log-likelihood: -521.9 (df = 15), AIC: 1074, BIC: 1123
#>
#> Coefficients:
#>                Estimate Std. Error
#> b1:(Intercept) 111.7977      4.044
#> b1:trt0.3       -1.4887      5.655
#> b1:trt0.5        9.2746      5.664
#> b1:trt0.6        9.9478      5.667
#> b1:trt0.8        7.6090      5.668
#> b2:(Intercept)  32.4085      0.871
#> b2:trt0.3       -1.1108      1.214
#> b2:trt0.5        0.1018      1.211
#> b2:trt0.6        0.4541      1.211
#> b2:trt0.8        0.6332      1.212
#> b3:(Intercept)  10.9504      0.151
#>
#> Random effects:
#>  Groups   Name Std.Dev. Corr
#>  plant_ID b1   8.711
#>           b2   1.784    -0.41
#>
#> Residual standard deviation: 2.277
#>
```

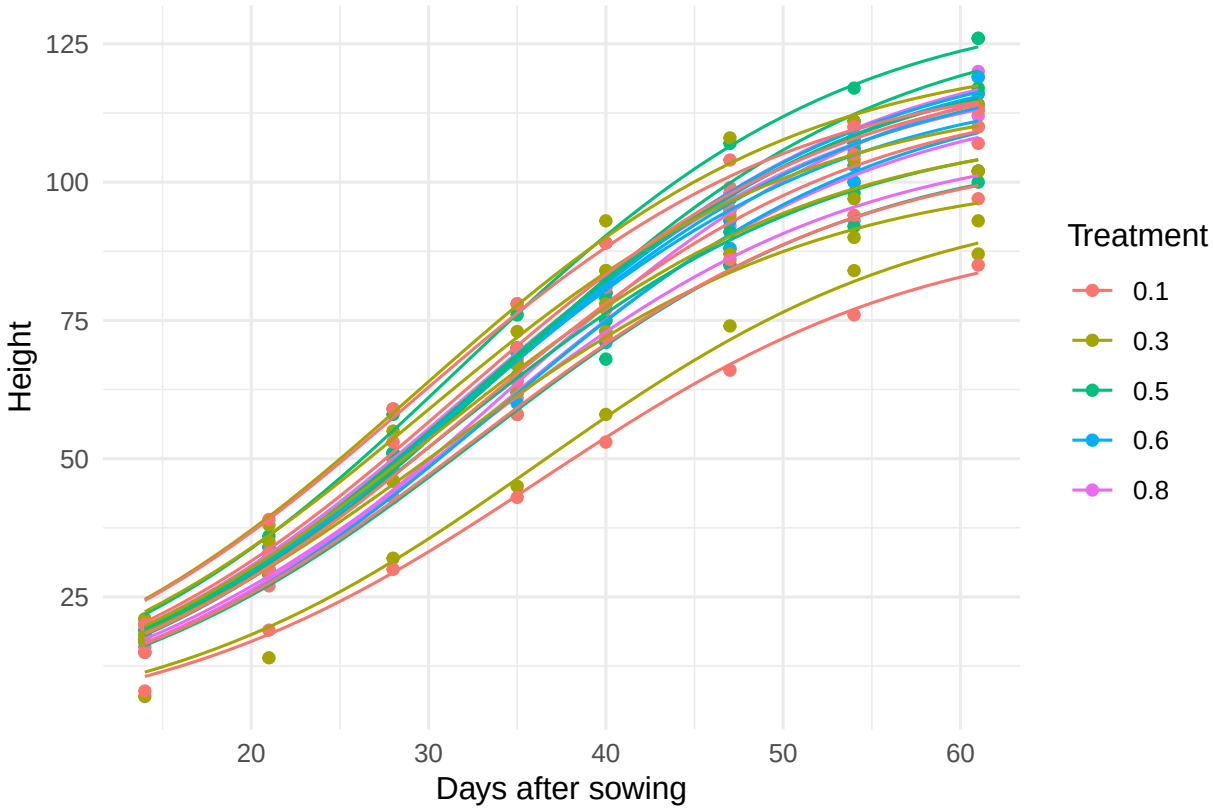
```
#> Optimiser: relative convergence (4), maximum absolute gradient 3.3e-05
```

Likelihood ratio tests by `anova()` compare nested models, here with and without the treatment effects.

```
fitre0 <- gcTMB(height ~ DAS, rform = b1 + b2 ~ 1 | plant_ID, data = FabaHeights25)
anova(fitre0, fitre)
#> Likelihood ratio tests of gcTMB models
#>
#>        npar    AIC    BIC  logLik deviance  Chisq Df Pr(>Chisq)
#> fitre0    7 1068.5 1091.6 -527.26   1054.5
#> fitre    15 1073.8 1123.3 -521.90   1043.8 10.705  8      0.219
```

`predict()` gives the curves of the fitted plants with their predicted random effects (`random = TRUE`), or the population curves of the treatments (`random = FALSE`); `timeto()` with `newdata` gives the times of the curves of each row.

```
nd <- unique(FabaHeights25[, c("plant_ID", "trt")])
nd <- merge(nd, data.frame(DAS = seq(14, 61, 1)))
nd$fit <- predict(fitre, newdata = nd)
ggplot(FabaHeights25, aes(x = DAS, y = height, colour = trt)) +
  geom_point() +
  geom_line(data = nd, aes(y = fit, group = plant_ID)) +
  labs(x = "Days after sowing", y = "Height", colour = "Treatment") +
  theme_minimal()
```



```
timeto(fitre, respLev = 90, newdata = data.frame(trt = levels(FabaHeights25$trt)), random = FALSE)
#> Times to reach the percentages of the asymptote:
#>
#>            Estimate Std.Error
#> 90:trt=0.1     56.5     0.981
#> 90:trt=0.3     55.4     0.972
#> 90:trt=0.5     56.6     0.976
#> 90:trt=0.6     56.9     0.979
#> 90:trt=0.8     57.1     0.981
```

Non-normal responses

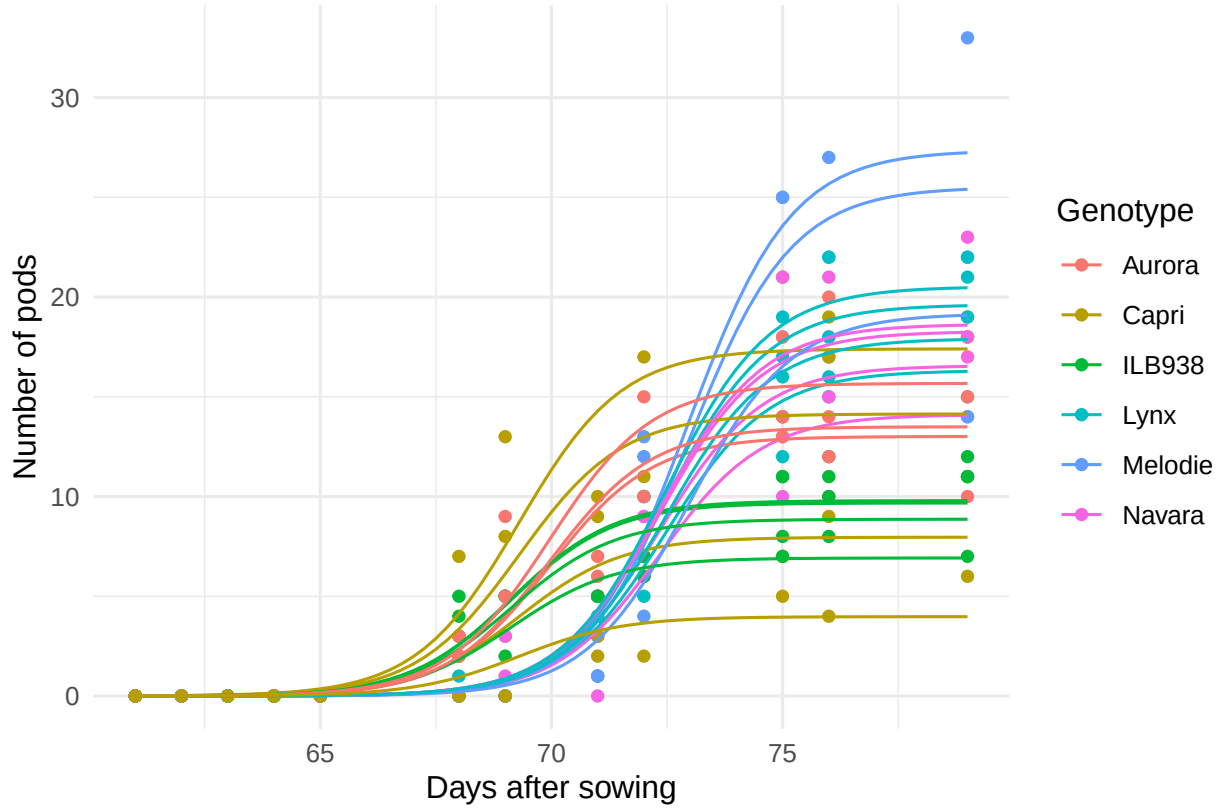
The `family` argument gives the distribution of the response: "gaussian", "lognormal", "poisson", "nbinom2", "nbinom1", "binomial", "betabinomial" or "beta". The data set `FabaCounts23` contains the numbers of pods of 22 faba bean plants of six genotypes, counted between 61 and 79 days after sowing. A Poisson model with an asymptote and a time of the maximum growth rate for each genotype, and a random asymptote for each plant:

```
fitp <- gcTMB(pods ~ DAS, fform = b1 + b2 ~ Genotype, rform = b1 ~ 1 | Tube_ID, data = FabaCounts23,
              family = "poisson")
summary(fitp)
#> Call:
#> gcTMB(form = pods ~ DAS, fform = b1 + b2 ~ Genotype, rform = b1 ~
#>     1 | Tube_ID, data = FabaCounts23, family = "poisson")
#>
#> Model:  logistic, increasing curve
#> Family: poisson, identity link for all parameters
#>
#> Number of observations: 263, groups: Tube_ID 22
#> Log-likelihood: -358.7 (df = 14), AIC: 745.3, BIC: 795.3
#>
#> Coefficients:
#>                     Estimate Std. Error
#> b1:(Intercept)       14.1865      2.241
#> b1:GenotypeCapri     -3.1804      2.872
#> b1:GenotypeILB938    -5.2359      2.855
#> b1:GenotypeLynx       4.5390      3.070
#> b1:GenotypeMelodie    9.9394      3.442
#> b1:GenotypeNavara     2.8472      3.038
#> b2:(Intercept)       69.9583      0.417
#> b2:GenotypeCapri     -0.6566      0.502
#> b2:GenotypeILB938    -0.8201      0.541
#> b2:GenotypeLynx       2.5084      0.452
#> b2:GenotypeMelodie    3.0039      0.455
#> b2:GenotypeNavara     2.3772      0.460
#> b3:(Intercept)        1.1138      0.103
#>
#> Random effects:
#>  Groups  Name Std.Dev.
#>  Tube_ID b1   3.269
#>
#> Pearson dispersion: 0.5838 (chi-square 145.4 on 249 df)
#>
```

```
#> Optimiser: relative convergence (4), maximum absolute gradient 7.5e-05
genotypes <- data.frame(Genotype = sort(unique(FabaCounts23$Genotype)))
predict(fitp, newdata = genotypes, type = "parameters", random = FALSE)
#>             b1       b2       b3
#> [1,] 14.186484 69.95825 1.113843
#> [2,] 11.006049 69.30168 1.113843
#> [3,]  8.950558 69.13818 1.113843
#> [4,] 18.725442 72.46665 1.113843
#> [5,] 24.125898 72.96216 1.113843
#> [6,] 17.033711 72.33549 1.113843
```

All the counts with the fitted curve of each plant, with its predicted random asymptote:

```
days <- seq(61, 79, 0.25)
ndp <- merge(unique(FabaCounts23[, c("Tube_ID", "Genotype")]), data.frame(DAS = days))
ndp$fit <- predict(fitp, newdata = ndp)
ggplot(FabaCounts23, aes(x = DAS, y = pods, colour = Genotype)) +
  geom_point() +
  geom_line(data = ndp, aes(y = fit, group = Tube_ID)) +
  labs(x = "Days after sowing", y = "Number of pods", colour = "Genotype") +
  theme_minimal()
```



By genotype, the curves of the plants together with the population curves of the genotypes (`random = FALSE`), the fixed effects, with 95% confidence intervals. For Poisson models, the confidence intervals of the mean are computed on the log scale, so that they stay positive.

```r
# the population curve of each genotype
ndg <- merge(genotypes, data.frame(DAS = days))
ndg <- cbind(ndg, predict(fitp, newdata = ndg, random = FALSE, interval = "confidence"))
ggplot(FabaCounts23, aes(x = DAS, y = pods)) +
  geom_ribbon(data = ndg, aes(x = DAS, ymin = lwr, ymax = upr), inherit.aes = FALSE, fill = "blue", alpl
  geom_line(data = ndp, aes(y = fit, group = Tube_ID), colour = "grey50") +
  geom_line(data = ndg, aes(y = fit), colour = "blue", linewidth = 1) +
  geom_point(alpha = 0.6) +
  facet_wrap(~ Genotype) +
  labs(x = "Days after sowing", y = "Number of pods") +
  theme_minimal()
```

